BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Rate of completed instructions.

Q2. Dependency/resource/control issue that interrupts pipeline.

Q3. Cache.

Q4. Q becomes0, sampled at the active edge.

You Do pages · Independent questions

Q1. I1 F1 D2 E3 [2], I2 F2 D3 E4 [2], I3 F3 D4 E5 [2].

Q2. Pipeline3+6−1=8 cycles [3], serial6×3=18 [2], equal stage/no stall assumption [1].

Q3. Data dependency waiting [2], branches flush/stall [2], average throughput drops though per-instruction latency not necessarily improved [2].

Exit Ticket · Model answer

False. It overlaps stages, improving throughput when conditions allow.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

LESSON 095 · Lesson 095 · CPU Pipelining: Overlap and Hazards

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is pipeline throughput?

Q2. What is a hazard?

Q3. Which CPU feature may reduce RAM accesses?

Q4. A D flip-flop has D=0 and Q=1 just before the active rising edge. What is Q after?

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

pipeline — Stages overlapping different instructions.

throughput — Completed work per time.

data hazard — Dependency on unavailable result.

control hazard — Branch uncertainty interrupting fetch.

New learning

Observe — Pipelining splits instruction processing into stages; several instructions can occupy different stages during the same clock cycle.

Explain — With four independent instructions and three one-cycle stages, ideal completion is 3+4−1=6 cycles; serial non-overlap takes 4×3=12.

Evaluate — Data dependencies, branching and resource conflicts can stall or flush a pipeline. Throughput benefits need ideal assumptions, not automatic latency reductions.

Worked example

Three instructions pass through a two-stage pipeline. Compute ideal completion cycles.

1. Explain the mechanism — 2+3−1=4 cycles; serial baseline 2×3=6 cycles. — The second task starts stage one while first advances to stage two.

We Do · Practise with guidance

Five independent instructions use a four-stage ideal pipeline. Calculate completion.

Explain the mechanism

4+5−1=8 cycles, compared with 4×5=20 serial stage cycles.

Assumes equal one-cycle stages, no resource conflicts or stalls.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Show cycles for I1-I3 in a 3-stage F-D-E pipeline, and finish cycle of I3. [6]

Question 2

Compute ideal pipeline versus serial completion for 6 instructions and 3 stages. [6]

Question 3

Explain why branch and data hazards can reduce actual improvement, distinguishing latency and throughput. [6]

Exit Ticket · Answer before checking

A pipeline makes each stage finish in less than one cycle by default.